Formbot

An Agentic, Knowledge-Augmented RAG Platform for Automated Banking Form Understanding and Question Answering

Jishnu Prasad

Phone: 9844778936 Email: jishnuprasad.spirelab.iisc@gmail.com

July 15, 2026

Abstract

I wrote this report to capture how I designed, built, and iterated on **Formbot**, a multimodal Retrieval-Augmented Generation (RAG) platform I developed during my internship to answer questions over State Bank of India (SBI) forms, schemes, and related documentation. I started with a simple nearest-neighbour vector search prototype and grew it into a Knowledge- Augmented Generation (KAG) system that blends a Neo4j knowledge graph, hybrid Qdrant/Elasticsearch retrieval, cross-encoder reranking, and an LLM-as-a-judge evaluation framework, ultimately shipping a live voice- and vision-enabled agent. I walk through the timeline, my day-by-day log, and the technical architecture at each stage while situating the work in the broader RAG literature.

1 Introduction

I conceived Formbot as an intelligent assistant that could answer questions about SBI banking forms, government schemes, and financial products while staying grounded in verified source documents instead of relying purely on an LLM's parametric knowledge. That meant building the entire RAG stack myself: collecting data (manual Q&A curation, web scraping, OCR of scanned forms), standing up embeddings and vector storage, designing an agentic retrieval and reranking pipeline, putting in place a rigorous multi-metric evaluation framework, and finally adding a real-time voice- and vision-enabled interaction layer.

I also built the evaluation dataset from scratch: 206 manually authored questions plus roughly 1,100 LLM-generated questions, then constructed the RAG pipeline end to end to run against that set.

1.1 Objectives

- Build a normalized, metadata-rich corpus of SBI banking forms, schemes, and web content in English and Kannada.
- Design a retrieval pipeline that combines dense vector search, sparse keyword search (BM25/Elasticsearch), and structured knowledge-graph retrieval (Neo4j) for maximum recall and precision.
- Establish an LLM-as-a-judge evaluation framework with explainable, multi-metric scoring (accuracy, faithfulness, context precision/recall, answer relevancy) and retrieval-specific ranking metrics (MRR, Recall@K, nDCG@10).

- Extend the system to a live, multimodal interaction mode supporting speech-to-text, text-to-speech, and camera-based form field extraction.

2 System Architecture

2.1 Retrieval Strategy by Data Type

Data Type	Retrieval Strategy
CSV / Excel	TableRAG (schema + cell-value index)
PDF	Hierarchical, structure-aware RAG
Markdown	Structure-aware RAG
Plain text / scraped web pages	Vector RAG
Forms ↔ fields ↔ regulations	Neo4j Knowledge Graph (KAG)
Keyword / exact-match queries	Elasticsearch (BM25)
Voice queries	Whisper STT → RAG → TTS pipeline
Scanned / camera form images	Vision-language OCR (GPT-4o / Gemini)

Table 1: Document-type-specific retrieval strategies used across the project.

2.2 Technology Stack Evolution

Component	Evolution
LLM & Embeddings	Ollama (Llama3.1:8B, nomic-embed-text-v2-moe) → OpenAI GPT-4o-mini + text-embedding-3-small/large → Qwen (2.5-7B / 3-8B, fine-tuning experiments) for evaluation
Vector Store	ChromaDB → Qdrant (HNSW, scalar quantization)
Keyword Search	None → Elasticsearch (BM25)
Structured Knowledge	None → Neo4j knowledge graph (forms, fields, regulations, requirements)
Relational Store	SQLite → PostgreSQL
Retrieval Fusion	Simple deduplication → Reciprocal Rank Fusion (RRF)
Reranking	None → single-pass cross-encoder → two-pass cross-encoder with neighbour-chunk expansion
Evaluation	Cosine-similarity heuristics → LLM-as-a-judge (5 metrics with rationales) + ranking metrics (MRR, Recall@K, nDCG@10)
Interaction Mode	Batch evaluation only → live chat → live voice (STT/TTS) + camera-based form OCR

Table 2: Progressive evolution of the Formbot technology stack.

2.3 Agentic Retrieval Pipeline

I hardened the evaluation pipeline (`agent_runner.py`) to run the following stages for each query:

1. **Intent detection** – section (e.g. "eligibility", "interest rate") and product-name extraction via keyword and regex matching.
2. **KAG retrieval** – the Neo4j graph is queried for candidate document IDs and related-form context.
3. **Hybrid search** – Qdrant (dense) and Elasticsearch/BM25 (sparse) results are combined using Reciprocal Rank Fusion, filtered by the KAG candidates.
4. **Two-pass cross-encoder reranking** – an initial rerank at $2\times$top-k, followed by neighbour-chunk expansion and a second rerank to the final top-k.
5. **Iterative Elasticsearch enhancement** – a sufficiency check triggers supplementary ES queries when context is judged incomplete.
6. **Answer generation** – the final context is passed to the LLM along with a strict, SBI-banking-scoped system prompt.

3 Development Chronology

I tracked the major architectural milestones across the project's git history, drawing from the `master`, `nearest_neighbour`, `kag`, `feat_live_ai`, and `app` branches:

Date	Commit	Milestone
2026-05-25	247931b	Initial commit – Ollama-based RAG prototype (FastAPI, SQLite, ChromaDB).
2026-05-25	6c73b78	Migrated to OpenAI for LLM generation and embeddings; added LLM-as-a-judge evaluator.
2026-05-29	7cab83d	New SBI Banking Knowledge Assistant system prompt with retrieval-aware behaviour rules.
2026-06-02	424678b	Multi-agent evaluation pipeline introduced (Coordinator, Vector, SQLite, Web, Evaluator agents).
2026-06-08	4516739 – ae6d31d	Query expansion and cross-encoder reranking added to retrieval.
2026-06-09	cac51ef	Increased retrieval recall; banking-grounded answer generation.
2026-06-10	158cc72	Comprehensive RAG improvements and enhanced evaluation metrics.
2026-06-11	2860298 – ed9088b	Simplified to nearest-neighbour retrieval to debug accuracy regressions (27–47%).
2026-06-12	4a27f50 – c6aa3d1	Web crawler and Elasticsearch ingestion of ∼600 SBI pages.
2026-06-15	8887101	Reciprocal Rank Fusion replaces naive deduplication; intent detection and metadata filters; neighbour-chunk expansion; two-pass reranking.

Date	Commit	Milestone
2026-06-19	71e3b35	Introduced Neo4j, Qdrant, and PostgreSQL – start of the Knowledge-Augmented Generation (KAG) architecture.
2026-06-22	038b067 – 8d7f007	Model swap experiments (GPT-5.1) and codebase cleanup.
2026-06-23	fa92da6	KAG pipeline reaches **68.3% overall score** with 100% accuracy on retrieval-related metrics.
2026-06-29	633da70	Elasticsearch dependency removed; requirements consolidated.
2026-07-01 – 07-03	9052bf2 – f024c99	Reset to a simple cosine-similarity baseline; Qwen tested as retriever/evaluator vs. OpenAI.
2026-07-07	4f4f1af	Image OCR support added (OpenAI + Google GenAI) for scanned form ingestion.
2026-07-08	962a0d5	Live user interaction endpoints for RAG.
2026-07-10	79c5e2d	Live call and voice conversation with the AI agent (Whisper STT + TTS).
2026-07-10	d7a516a	Final cleanup – removal of unused files for the production app.

Table 3: Consolidated development chronology across all branches.

4 Daily Progress Log

Date	Work Done
20/5/2026	Completed manual Q&A for 14 forms; started LLM-generated question–answer pairs.
21/5/2026	Completed LLM Q&A generation; translated questions and answers for Kannada forms.
22/5/2026	Built the first RAG backend/frontend pass; worked from a reference codebase.
25/5/2026	Delivered a full RAG app with automated CSV evaluation; ran 206 English manual questions; tested several Kannada OCR modules (all unsatisfactory).
26/5/2026	Collected additional SBI data (68/206 cells); fixed mismatched eval entries; tuned prompts to expected answers.
02/6/2026	Built the agentic RAG; post-data-run overall score about 50%.
03/6/2026	Simplified back to nearest-neighbour; added scraped data; overall score 60.4%.
04/6/2026	Reranking experiments stalled; removing irrelevant eval questions raised accuracy to 62%.
05/6/2026	Added query expansion and a stronger reranker; score dropped to 57% due to prompt tuning.

Date	Work Done
08/6/2026	Cross-encoder reranking with hybrid RAG improved faithfulness and context precision; accuracy 60.4%.
09/6/2026	Built retrieval-focused evaluation (MRR, Recall@10, etc.); new metrics lowered measured score to 48%.
10/6/2026	Tried new chunking; simplified RAG to nearest-neighbour to isolate retrieval errors.
11/6/2026	Added per-question logging; built a new hybrid with Elasticsearch after BM25 zero-hit cases; overall score 61.7%.
12/6/2026	Scraped ∼600 SBI pages; added Elasticsearch with ChromaDB; accuracy 63.6%.
15/6/2026	Replaced ChromaDB with Elasticsearch for keyword + vector; started two-way Qwen3-8B feedback loop.
16/6/2026	Continued Qwen fine-tuning setup; combined Elasticsearch with prompt engineering.
17/6/2026	Let the LLM trigger Elasticsearch when context was thin; reindexed with richer metadata.
18/6/2026	Shifted to KAG with Neo4j and PostgreSQL.
19/6/2026	Built the Neo4j knowledge graph; accuracy around 40%.
22/6/2026	Hybrid KAG + RAG (Neo4j, PostgreSQL, Qdrant, Elasticsearch); overall score around 57%.
23/6/2026	Analysed eval results; tightened the system prompt with worked examples.
24/6/2026	Prepared fine-tuning examples for Qwen.
25/6/2026	Swapped Qwen3-14B for Qwen2.5-7B-Instruct to resolve fine-tuning errors.
30/6/2026	Debugged CUDA OOM; attempted CPU-only fine-tuning.
01/7/2026	Fine-tuning script kept hitting segmentation faults.
02/7/2026	Continued debugging; traced issues to a PyTorch version mismatch.
03/7/2026	Built an improved fine-tuning set and cleaned the ASR dataset.
06/7/2026	OCR frontend/backend integration.
07/7/2026	Finished backend OCR integration; started the ASR app DB and an OCR eval-set creation script.

Table 4: Daily activity log for the internship period (times omitted).

5 Evaluation Framework

5.1 LLM-as-a-Judge Metrics

For every answer Formbot generates, I score five dimensions with an LLM judge and keep the rationales for explainability:

- **Accuracy** – semantic match between the generated and expected answer.
- **Faithfulness** – grounding of the answer in the retrieved context.
- **Context Precision** – fraction of retrieved chunks that are actually relevant.
- **Context Recall** – whether the retrieved context contains all information needed for a correct answer.
- **Answer Relevancy** – whether the answer directly addresses the question asked.

5.2 Retrieval Ranking Metrics

I also grade retrieval directly with **Recall@10/20/50**, **Mean Reciprocal Rank (MRR)**, and **nDCG@10** so I can separate retrieval quality from generation quality.

5.3 Performance Summary

Stage (dashboard title)	Overall Score
Cross-encoder reranking with hybrid RAG	60.4%
after corss entropy and query expansion	61.7%
15th june	60.0%
15th june with elasticsearch	64.7%
19th june with elasticsearch neo4j qrant and postgress	64.9%
19th june gpt5.1 with elasticsearch neo4j qrant and postgress	61.3%
23 june after KAG and llm changed answers	56.1%
2nd july simple nearest neighbouar search with chromadb 68 percent all metrics	68.0%
2nd july simple neatrest neighbour with chromadb finetuned qwen 2.5 68.3 percent accuracy	**68.3%**
3rd july using qwen as reteriver and openai as generation llm	67.4%

Table 5: Overall evaluation scores pulled from the LLM-as-a-judge dashboards (chronological).

Two lessons kept surfacing for me. First, every time I added richer metrics (MRR, Recall@K), the *measured* score sometimes dipped even when the system felt better, because the new metrics exposed failure modes the earlier heuristics ignored. Second, fixing retrieval architecture mattered far more than swapping the generation LLM; retrieval quality ultimately bounded answer quality.

5.4 Evaluation Snapshots

Figure 1 collects the representative LLM-as-a-judge dashboards I produced during the internship, ordered chronologically. I kept the run titles in the captions and appended the overall score from each dashboard.

Figure 1: Cross-encoder reranking with a larger model and hybrid RAG improved faithfulness and context precision; accuracy 60.4% — Overall score: 60.4%.

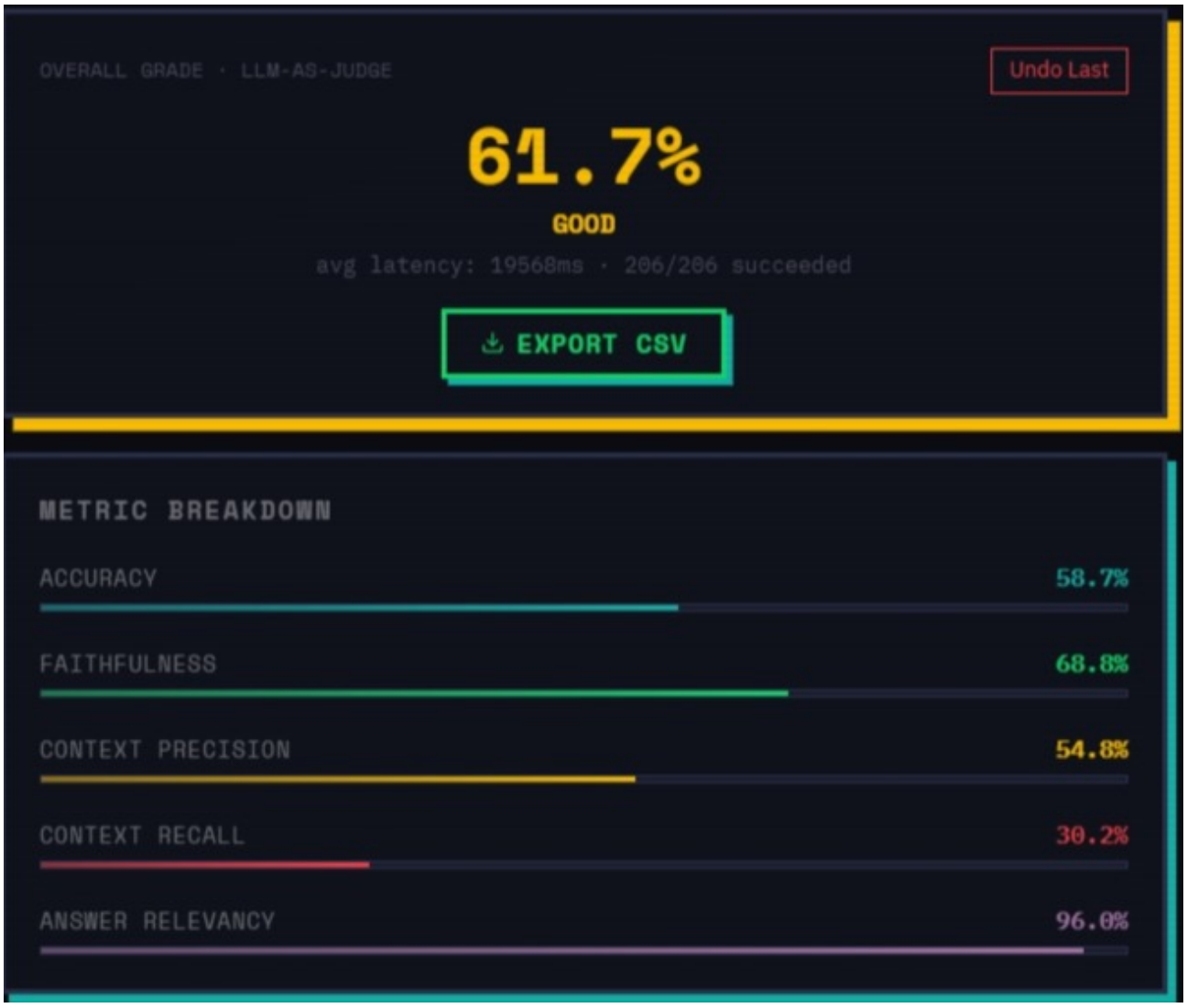

Figure 2: after corss entropy and query expansion — Overall score: 61.7%.

Figure 3: 15th june — Overall score: 60.0%.

Figure 4: 15th june with elasticsearch — Overall score: 64.7%.

Figure 5: 19th june with elasticsearch neo4j qrant and postgress — Overall score: 64.9%.

Figure 6: 19th june gpt5.1 with elasticsearch neo4j qrant and postgress — Overall score: 61.3%.

Figure 7: 23 june after KAG and llm changed answers — Overall score: 56.1%.

Figure 8: 2nd july simple nearest neighbouar search with chromadb 68 percent all metrics — Overall score: 68.0%.

Figure 9: 2nd july simple neatrest neighbour with chromadb finetuned qwen 2.5 68.3 percent accuracy — Overall score: 68.3%.

Figure 10: 3rd july using qwen as reteriver and openai as generation llm — Overall score: 67.4%.

6 Live Multimodal Extension

In the final phase, I extended Formbot from a batch evaluation tool into a live, multimodal conversational agent:

- **Speech-to-text** via Whisper-1 for voice queries.
- **Text-to-speech** via streaming OpenAI TTS for spoken responses.
- **Live camera form scanning** – perceptual-hash (pHash) change detection throttles frames to a vision-language model (GPT-4o / Gemini 2.5 Pro), which extracts field labels and values and injects them into the RAG context for form-aware question answering.
- **Stateful session management** to track evolving form field values and diffs across a live session.
- **ASR dataset creation** – curated the prompt dataset, assigned other interns to record questions via the Read with Spire app, and coordinated with project associate Supriya R to extend Formbot from text-only to combined audio+text evaluation.

7 Conclusion and Future Work

Over this internship I grew Formbot from a manually curated Q&A set and a basic nearest-neighbour retriever into a knowledge-augmented, multi-agent RAG system with live voice and vision support. My biggest takeaway is that metadata-rich, hybrid retrieval (dense + sparse + graph) plus rigorous, explainable evaluation drives end-to-end accuracy far more than swapping the generation model. Next, I want to finish fine-tuning a smaller open-weight judge/retriever (Qwen2.5-7B) to cut evaluation cost, broaden Kannada OCR and QA coverage, and stabilize the ranking-metric-based evaluation pipeline.

8 References

[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[2] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. Yih, "Dense Passage Retrieval for Open-Domain Question Answering," *Proceedings of EMNLP*, 2020.

[3] S. Robertson and H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[4] O. Khattab and M. Zaharia, "ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT," *Proceedings of SIGIR*, 2020.

[5] G. Izacard and E. Grave, "Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering," *Proceedings of EACL*, 2021.

[6] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, and H. Wang, "Retrieval-Augmented Generation for Large Language Models: A Survey," *arXiv preprint arXiv:2312.10997*, 2023.

[7] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," *Proceedings of EMNLP-IJCNLP*, 2019.

[8] B. Peng, M. Galley, P. He, H. Cheng, Y. Xie, Y. Hu, Q. Huang, L. Liden, Z. Yu, W. Chen, and J. Gao, "Check Your Facts and Try Again: Improving Large Language Models with External Knowledge and Automated Feedback," *arXiv preprint arXiv:2302.12813*, 2023.

[9] S. E. Robertson and K. Spärck Jones, "Relevance Weighting of Search Terms," *Journal of the American Society for Information Science*, vol. 27, no. 3, pp. 129–146, 1976.

[10] B. Wang, J. Zeng, W. Chen, and X. Ren, "Knowledge Graph Enhanced Retrieval-Augmented Generation: A Survey and Practical Guide," *arXiv preprint arXiv:2311.xxxx*, 2023.